

Heterogeneous computing to accelerate stochastic biochemical network simulation and inference

Luca Lubrano Lavadera

Under the supervision of:

Dr. David J. Warne

October 26, 2025

Abstract

Stochasticity underpins many biological processes within biochemical systems and the wider life sciences. Often, we wish to examine these systems and perform statistical inference to calibrate and estimate model parameters to real world data. In doing so, we are required to generate a large amount of simulation data and perform heavy computational tasks. It is estimated that computational research and high performance computing (HPC) contribute a non-trivial and increasing amount to green house gas (GHG) emissions. A substantial source of the emissions caused by GHG is the energy requirements needed to compute sustained data intensive tasks in these HPC systems. Field programmable gate arrays (FPGAs) are reconfigurable computing devices that are capable of low-power high-throughput and may be able reduce the power usage in both the active draw and cooling requirements of HPC. We present several well established mathematical methods for simulating biochemical reaction networks using a portable OpenCL implementation. These methods display significant improvements in both the performance of methods and power consumption criteria. Further power efficiency could be generated by utilising reconfigurable computing hardware in statistical inference.

1 Introduction

Systems biology looks to capture the complex behaviour of dynamical biochemical networks (BCRN) at an intracellular level. In preference to understanding the exact behaviour of individual molecular reactions, systems biology instead takes a macroscopic approach to better understand and model the functioning of cells as a whole [1–4]. Many living cells however, often contain very low copy numbers of important molecules [5, 6]. It is well accepted that this plays a crucial role in introducing stochasticity into biological processes [3, 6, 7].

To understand cellular systems, we need to be able to simulate and calibrate complex stochastic models to experimental data [4, 8]. The classical approach to solving this problem in a Bayesian setting requires the likelihood function that provides the probability of the data under assumption of a mathematical model given known parameters. For biochemical reaction network models, the likelihood depends on the solution to either the chemical master equation (CME), the chemical Langevin equation (CLE), or reaction rate equations (RRE). This involves either solving a *very large* system of ODE’s — one for each possible system state, a set of coupled SDE’s — one for each species — or an ODE system respectively.

In reality many biochemical systems are analytically intractable, meaning we must rely on computationally expensive stochastic simulations to make progress using simulation based inference (SBI) methods [4, 9]. These inference methods require an *enormous* amount of these simulations to produce meaningful results. This leads to the requirement of potentially power inefficient high performance computing (HPC) systems to obtain timely results. The push for ever-more computation performance has lead to HPC often considering power consumption as a secondary effect. However, recent studies into the green house gas (GHG) emissions caused by computational research has found that the sector produces a non-trivial percentage of emissions [10–12]. Green high performance computing (GreenHPC) aims to tackle the challenge of highly power efficient compute [13, 14]. Unfortunately,

this is mostly done through the use of accelerator hardware, such as application specific integrated circuits (ASICs), which present significant programming challenges [13].

To address the competing goals of improved time-to-result and lower-power consumption, we consider heterogeneous computing implementations of simulation and inference methods. This aims to both improve simulation time and reduce power consumption of large systems biology inference problems that will likely be required for the next generation of whole cell models. The development of reconfigurable computing and code compilation targeting these architectures has allowed these systems to be used extensively in GreenHPC [13]. This has meant the accessibility of more complex algorithms to be developed to target the reconfigurable architecture [15, 16].

1.1 Research Questions

The key aim of this work is to enable SBI to be scalable to large scale chemical reaction networks, both in terms of computational burden and power consumption. To this end, we consider the following research questions.

1. How can heterogeneous computing approaches improve the scalability of simulations and inference in systems biology in high performance computing?
2. To what extent can reconfigurable computing approaches improve power efficiency for green high performance computing?

1.2 Contribution

We aim to achieve low-power compute by implementing common stochastic simulation algorithms (SSAs) for biochemical reaction networks in the portable OpenCL programming language. Through this programming model, we are able to compile our kernels to CPU, GPU or FPGA architectures. We are able to achieve a reduction in wall-time of up to 80 times as compared to the Julia implementations — a common programming language for mathematical modelling — and a reduction in power usage of approximately 170 times. We expect that as we move to FPGAs we will be able to reduce the power consumption further through task specific configurations of the reconfigurable computing hardware.

2 Literature Review

In this section we review key background topics required to appropriately understand the works completed through this paper. There is an extensive breadth of literature discussing whole cell modelling and statistical inference. As such, we narrow our review of topics to specifically biochemical reaction networks and their related SBI methods.

2.1 Systems Biology

The classical approach to describing biological systems are continuous-time deterministic models [4, 8]. Using deterministic models however, fails to explore the intrinsic randomness of biochemical reaction networks, and as such we lose valuable information about the true nature of the system [6, 7]. As a result of this, the development of stochastic methods have become popular to capture the variability among genetically identical cells [7].

Gene expression and regulation are fundamental applications of stochastic modelling methods in which research aims to understand the processes of cellular reproduction and phenotypic characteristics [3, 6]. It is well established that these processes are fundamentally stochastic. The nature of experimental data within whole cell modelling introduces substantial complication in generating and fitting stochastic models [4]. As such, it is common to instead generate stochastic simulations to produce meaningful data for analysis [8, 17].

Within systems biology, the two computational challenges we define are the forwards and inverse problems [9]. The forwards problem refers to the challenge of predicting the future state of a system. Namely we must consider the simulation of sample paths of our biochemical reaction networks over

time, and the expectation of average behaviour among all possible reaction events [9]. The inverse problem is that of inference, since experimental data is often measured using fluorescent marker proteins we are unable to yield full-picture results. This is because these fluorescent reporter genes are often only able to be attached to a small number of species and can become indistinguishable in observations [18, 19]. Bayesian methods are thus underpinned on using simulation based methods to generate time-course data [4].

2.2 Inference Methods

In real world applications of systems biology modelling, we often are interested in performing statistical inference to estimate reaction rate parameters, known as the *inverse problem*. A popular choice to solve this problem would be Bayesian inference, being well tested and heavily used in the wider life science field [20–22]. Given some observed state, $\mathbf{Y}_{\text{obs}}$, and a BCRN with kinetic rate parameters, $\boldsymbol{\theta} = [k_1, \dots, k_M]^T$, we define Bayes theorem to be

$$p(\boldsymbol{\theta}|\mathbf{Y}_{\text{obs}}) = \frac{p(\mathbf{Y}_{\text{obs}}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathbf{Y}_{\text{obs}})},$$

where $p(\boldsymbol{\theta}|\mathbf{Y}_{\text{obs}})$ is the posterior, $p(\mathbf{Y}_{\text{obs}}|\boldsymbol{\theta})$ is the likelihood, $p(\boldsymbol{\theta})$ is the prior and $p(\mathbf{Y}_{\text{obs}})$ is the evidence. Unfortunately evaluating the likelihood function for stochastic biochemical networks requires the solution to the chemical master equation (CME) which becomes analytically intractable for even simple problems [9, 23–25]. We therefore often defer to simulation based inference (SBI) methods to solve this problem.

Approximate Bayesian computation (ABC) is an example of a traditional SBI method that utilises Bayesian statistics to remove the need for an analytical expression of the likelihood. All ABC methods utilise simulations in order to approximate the likelihood function, which are subsequently compared to observed data. Most commonly, this is used to calculate a Euclidean distance between our simulated data and true data, which is then evaluated to determine whether the prior is *close enough* to our observed data based on a discrepancy tolerance, $\epsilon > 0$ [26]. For accurate inference we require ϵ to be sufficiently small, leading to substantial computational challenges as we increase the dimensionality of our system, the probability that our prior will generate a dataset that is sufficiently close to the real data decreases [9, 17]. While these methods are well-founded in mathematical theory, the calibration and assumptions made when using these models needs to be carefully considered to derive accurate results [9, 17, 26].

More advanced SBI methods have also been developed, aiming to tackle pitfalls in the ABC approach [9, 17]. So called Neural SBI methods utilise machine learning (ML) algorithms, particularly deep neural networks, to allow modellers to work with higher dimensional data [27]. This has also been expanded with Sequential Neural Likelihood methods, a likelihood-free method based on training a ML model to estimate the conditional probability density of data to model the likelihood [28]. Other methods attempt to reduce the data complexity by using Normal approximations of the distribution of a set of summary statistics [29, 30]. As well as Synthetic likelihood methods which use noisy, unbiased estimates of the target distribution in a MCMC scheme to sample from the exact target distribution [31].

2.3 Green High Performance Computing

Most fields in modern science rely heavily on data intensive high performance computing (HPC). Server based computation — of which computational research plays a large part — is estimated to create more greenhouse gas (GHG) emissions than the aviation industry [11]. This impact is only expected to increase as tools, such as statistical machine learning, gain popularity within the field [32, 33]. To help solve this issue, we require efficient algorithmic practices as well as low-power hardware. Specifically, we require algorithms that both reduce wall-time and are also efficient in their use of physical hardware. The most significant contributor to this impact from data centres is power draw during computation, and subsequently the cooling systems required to maintain workable temperatures for the servers [34, 35].

It follows that in order to reduce energy consumption, we require efficient computation hardware while retaining throughput to keep up with ever increasing workload. A technology that could answer this demand are field programmable gate arrays (FPGAs), devices which are able to reconfigure their hardware, enabling substantial power efficiencies over their CPU and GPU counterparts [36, 37]. While FPGAs are a well established tool, their viability for general computing has become an emerging area of research as compilation tools for higher level languages become accessible [15, 36, 38, 39].

2.3.1 Heterogeneous Computing

Heterogeneous computing is a high performance computing (HPC) framework in which multiple devices from a variety of vendors are used to maximise compute performance of the overall system. Often these systems combine CPUs, GPUs and various acceleration devices such as FPGAs, Intelligence processing units (IPUs) and application-specific integrated circuits (ASICs) with the goal to use each device for its most optimal application [40].

By combining these different compute architectures, a vast majority of which being CPU-GPU [41, 42], heterogeneous computing focuses unequivocally on performance [43, 44]. Modern CPU's have tens of high-frequency transistor dense cores, which typically perform out-of-order, multi-instruction computes. This makes them especially suitable for low-latency tasks with complex control flows. Comparatively, GPU's use a substantially larger number of lower frequency cores to process uniform control flow operations [43]. This makes the GPU ideal for high-throughput, simple, repetitive tasks such as graphics rendering. Thus, by utilising both devices, a HPC system is able to perform a much larger array of tasks and provide better performance as compared to homogeneous processor systems [43].

2.3.2 Reconfigurable Computing

Reconfigurable computing is classified by a computer architecture that is able to change its logic programming. The most common and popular example of which are FPGAs, built from 'reconfigurable fabric' which contains the configurable logic blocks, interconnections and BRAM that allow the hardware to change its logic functions [45]. In computing, this means that we are able to construct programs not at a software level, but instead program them onto the physical architecture of the card. This is a key aspect of the power efficiency that these devices can provide. Unlike CPUs and GPUs, they do not have any overhead power cost related to unnecessary and always active processes.

As a result of the freedom provided by FPGA hardware, we are also able to increase computational efficiency in low precision settings through the creation of unique precision levels. Unlike modern CPUs which can provide either double (64-bit), float (32-bit) or half-precision (16-bit), we are able to define custom bit length precision data types. This can be directly applied in areas such as multi-fidelity Monte-Carlo simulation, which sample cheap, low-precision simulations for Bayesian inference acceleration.

These devices have been traditionally used by electrical engineers for hardware prototyping and development, requiring users to understand low-level hardware description languages (HDLs). Making them inaccessible to scientific researchers without the proper background to fully take advantage of their utility. Recent advancements in compilation abilities have lead to the ability for OpenCL to be compiled into bit-stream and hence are now able to be used to program these devices. This has made them a target of active research in areas such as high-dimensional linear algebra [16, 36, 37, 39, 46], simulation problems [47] and time-critical safety operations [15].

Intelligence processing units (IPUs) are an emerging technology driven by the need for more specialised Artificial Intelligence computing capabilities [48]. Unlike traditional GPU architecture which uses single-instruction multiple data (SIMD) vectorised parallelism to accelerate tasks, the IPU utilises multi-instruction multiple data (MIMD) [49]. This allows each thread to perform operations in a distinct flow, and irregular data access patterns without loss of efficiency [49]. Similarly to FPGAs, these devices are also able to define custom precision data types, which has made them extremely efficient processors in the AI inference problem [50–52].

2.3.3 Mixed Precision Computing

FPGAs are also seeing use with Multilevel Monte Carlo (MLMC) methods in the financial sector [53–56], with algorithms such as that proposed by Haas and Giles [38]. Which exploits the ability to generate custom precision on hardware to perform Monte Carlo simulations within MLMC where the targeted accuracy is relatively low without losing accuracy of the overall approximation [38]. While these methods can be applied to CPU/GPU computing, they are limited to predefined floating point precisions — being 32-bit (single) or 64-bit (double). FPGAs however, are able to be programmed to utilise custom precision vector registers, allowing for much larger potential benefits to the proposed method in [38].

Classical ABC inference methods are built on a large amount of Monte Carlo sampling of the posterior distribution [17]. To tackle this, sampling schemes have been developed which utilise mixed accuracy simulations to reduce the cost of computing summary statistics for the inverse problem. MLMC is built on the principle that we wish to compute the expectation of a random variable by utilising lower accuracy simulations [57–66]. The generalised telescoping summation of the MLMC approximations can be written as

$$\mathbb{E}[P_L] = \mathbb{E}[P_0] + \sum_{l=0}^L \mathbb{E}[P_l - P_{l-1}], \quad (1)$$

where $P_0, \dots, P_{L-1}$ are a sequences of approximations to P_L with increasing accuracy, and hence increasing cost. This method exploits the varying levels of accuracy to reduce the overall simulation cost of the expectation, achieving computational speed up in SBI applications [57, 58].

3 Methodology

To begin implementing the proposed stochastic simulation, we first formalise mathematical derivation of the three methods used; Tau leaping, Gillespie direct, and Euler-Maruyama. We then describe the heterogeneous programming model of OpenCL and the unique terminology required to understand the proposed algorithms. We also describe in detail an example kernel of the Tau leaping method in OpenCL to describe the processes used to construct our implementations.

3.1 Stochastic Simulation Algorithms

To understand the overall behaviour of biochemical reaction networks (BCRN), we concern ourselves with only the change in population over time of the molecular species [4, 8, 9, 25]. We consider a spatially homogeneous system for the purposes of this work, however, all of the methods we present can be extended to the spatially heterogeneous case [67].

Consider a BCRN with the N unique chemical species, that interact according to M distinct chemical reactions. The j^{th} reaction is denoted by

$$\sum_{i=1}^N \nu_{i,j}^- X_i \rightarrow \sum_{i=1}^N \nu_{i,j}^+ X_i, \quad (2)$$

where $\nu_{i,j}^-$ and $\nu_{i,j}^+$ are the reactant and product stoichiometries related to the i^{th} chemical species in the j^{th} reaction. We denote $X_i(t)$ as the copy number, that is the number of molecules in the system, of the i^{th} chemical species at time $t > 0$. This leads to the system state vector, $\mathbf{X}(t) = [X_1(t), \dots, X_n(t)]^\top$. In the event that the j^{th} reaction occurs, the instantaneous system update is, $\mathbf{X}(t) = \mathbf{X}(t^-) + \boldsymbol{\nu}_j$, where $\mathbf{X}(t^-)$ is the state of the system immediately preceding time t . The stoichiometric vector $\boldsymbol{\nu}_j = \boldsymbol{\nu}_{*,j}^+ + \boldsymbol{\nu}_{*,j}^-$ defines the net change in the system state as a result of the j^{th} reaction event.

We set the kinetic rate parameters of the M chemical reactions to be the vector $\boldsymbol{\theta} = [k_1, k_2, \dots, k_m]$, where the j^{th} reaction is associated with parameter k_j . The occurrence of these reactions are based defined using the probability framework laid out in [24, 25]. For each reaction j , we have an associated *propensity function*, $a_j(\mathbf{X}(t); \boldsymbol{\theta})$. This gives the probability of reaction j occurring exactly once within

the infinitesimal interval $[t, t+dt)$ to be proportional to $a_j(\mathbf{X}(t); \boldsymbol{\theta})$, and $a_0(\mathbf{X}(t); \boldsymbol{\theta}) = \sum_{j=1}^M a_j(\mathbf{X}(t); \boldsymbol{\theta})$ [8, 9].

3.1.1 Gillespie Direct Method

The Gillespie direct method is based on constructing a numerical realisation of $X(t)$ from the chemical master equation

$$\frac{\delta P(\mathbf{X}, t | \mathbf{X}_0, t_0)}{\delta t} = \sum_{j=1}^M [a_j(\mathbf{X}(t) - \mathbf{v}_j; \boldsymbol{\theta}) P(\mathbf{X} - \mathbf{v}_j, t | \mathbf{X}_0, t_0) - a_j(\mathbf{X}(t); \boldsymbol{\theta}) P(\mathbf{X}, t | \mathbf{X}_0, t_0)]. \quad (3)$$

As previously mentioned in [Section 2.2](#), since the CME is a set of coupled ODE's, we rarely able to find an analytical solution. We instead focus on generating simulated paths for $X(t)$ using a new probability function $p(\tau, j | \mathbf{X}, t)$, the probability that the next reaction in the system will occur in the infinitesimal time interval $[t, t + \tau)$ will be reaction j . This function is then the joint probability density of the two random variables; time until the next reaction τ and the index of the next reaction j , given the current state of the system $X(t)$

$$p(\tau, j | \mathbf{X}, t) = a_j(\mathbf{X}(t); \boldsymbol{\theta}) \exp(-a_0(\mathbf{X}(t); \boldsymbol{\theta})\tau), \quad (4)$$

and $a_0(\mathbf{X}(t); \boldsymbol{\theta}) = \sum_{j=1}^M a_j(\mathbf{X}(t); \boldsymbol{\theta})$. τ is an exponential random variable $\tau \sim \text{Exp}(1/a_0(\mathbf{X}(t); \boldsymbol{\theta}))$ and j is an independent integer random variable $P(J = j) = a_j(\mathbf{X}(t); \boldsymbol{\theta})/a_0(\mathbf{X}(t); \boldsymbol{\theta})$, for $j \in \{1, 2, \dots, M\}$. This gives an exact realisation of the reaction path of $\mathbf{X}(t)$ [25, 68]. From this definition, we construct the algorithmic approach to the GDM described in [Figure 1](#).

Algorithm 1 Gillespie Direct Method

```

 $t \leftarrow 0$ 
 $\mathbf{X} \leftarrow \mathbf{x}_0$ 
while  $t < T$  do
    Calculate individual propensities,  $a_1(\mathbf{X}(t); \boldsymbol{\theta}), \dots, a_M(\mathbf{X}(t); \boldsymbol{\theta})$ 
    Calculate total propensity  $a_0(\mathbf{X}(t); \boldsymbol{\theta}) = \sum_{j=1}^M a_j(\mathbf{X}(t); \boldsymbol{\theta})$ 
    Generate time step  $\tau \sim \text{Exp}(1/a_0(\mathbf{X}(t); \boldsymbol{\theta}))$ 
     $j \leftarrow$  random integer  $P(J = j) = a_j(\mathbf{X}(t); \boldsymbol{\theta})/a_0(\mathbf{X}(t); \boldsymbol{\theta})$ , for  $j \in \{1, 2, \dots, M\}$ 
     $\mathbf{X} \leftarrow \mathbf{X} + \nu_{*,j}$ 
     $t \leftarrow t + \tau$ 
end while

```

3.1.2 Tau Leaping Method

While the exact solutions of the GDM SSA can be statistically important in certain analyses, the exactness of the solution potentially comes at a high computational price [24]. In order to reduce this cost, we must reduce the level of detail at which we model the system, such that our new methods become *approximate SSAs*.

Performing a numerical approximation to the trajectory representation of the CME using the Kurtz representation yields discrete-time Poisson process

$$\mathbf{X}(t) = \mathbf{x}_0 + \sum_{j=1}^M Y_j \left(\int_0^t a_j(\mathbf{X}(\eta(s)); \boldsymbol{\theta}) ds \right) \boldsymbol{\nu}_k, \quad (5)$$

where $\eta(s) = t_n$ if $t_n \leq s < t_{n+1}$ and Y_j are independent, unit-rate Poisson processes [69]. We are then able to discretise the integral in [Equation 5](#) using the Euler method, reducing the Poisson processes to Poisson random variables over $[t, t + \tau)$

$$\mathbf{X}(t + \tau) = \mathbf{X}(t) + \sum_{j=1}^M \boldsymbol{\nu}_{*,j} K_j(\mathbf{X}(t); \tau), \quad (6)$$

where $K_j(\mathbf{X}(t); \tau) = \mathcal{P}(a_j(\mathbf{X}(t); \boldsymbol{\theta})\tau)$ [24].

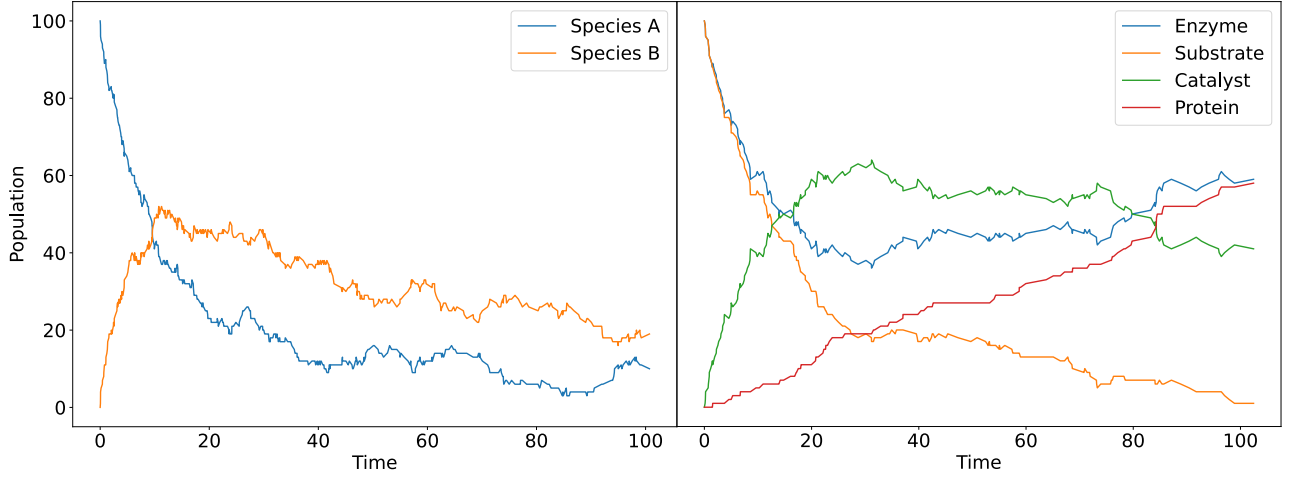


Figure 1: (left) Gillespie direct method sample path of the mono-molecular chain model using Julia on CPU. With kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$, initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$. (right) Gillespie direct method sample path of the Michaelis-Menten enzyme kinetic chain model using Julia on CPU. With kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$, initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$

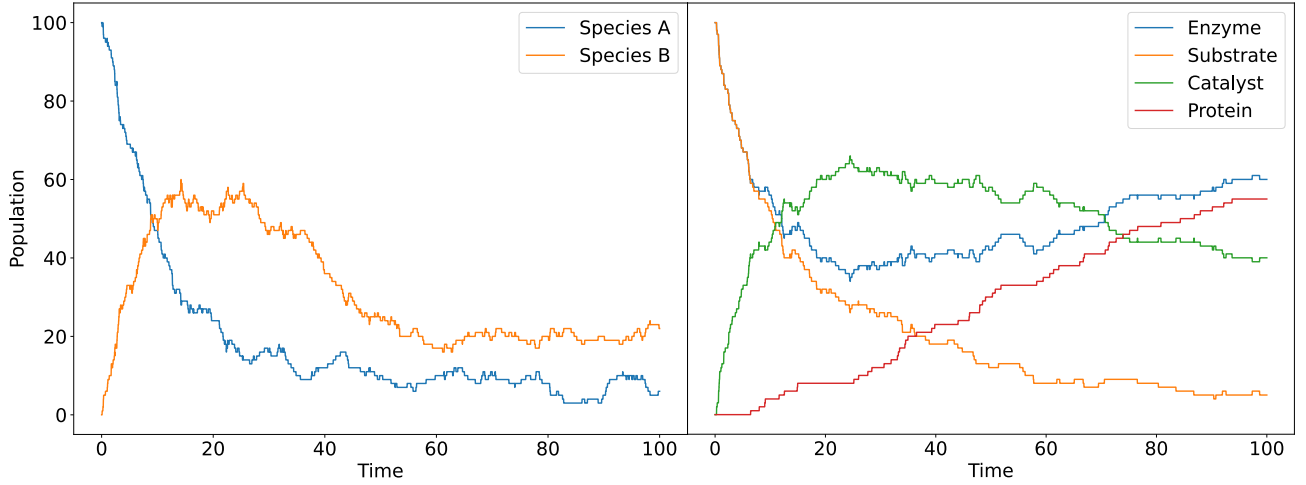


Figure 2: (left) Tau leaping method sample path of the mono-molecular chain model using Julia on CPU. With kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$, initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$. (right) Tau leaping method sample path of the Michaelis-Menten enzyme kinetic chain model using Julia on CPU. With kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$, initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$

3.1.3 Euler-Maruyama Method

The Euler-Maruyama method directly builds from the Ito stochastic differential equation (SDE) form of the chemical Langevin equation (CLE) [24].

$$X_i(t) = X_i(0) + \sum_{j=1}^M \nu_{i,j} \int_0^t a_j(\mathbf{X}(\eta(s)); \boldsymbol{\theta}) ds + \sum_{j=1}^M \nu_{ij} \int_0^t \sqrt{a_j(\mathbf{X}(\eta(s)); \boldsymbol{\theta})} dW_j(s), \quad (7)$$

where $W_j(t)$ are independent Wiener processes. We are then able to directly apply a Euler-Maruyama discretisation scheme to Equation 7 and recover our method.

$$dX_i(t) = \sum_{j=1}^M \nu_{i,j} a_j(\mathbf{X}(t); \boldsymbol{\theta}) dt + \sum_{j=1}^M \nu_{ij} \sqrt{a_j(\mathbf{X}(t); \boldsymbol{\theta})} dW_j(t) \quad (8)$$

We can further impose a restriction on τ such that the expected number of chemical reactions in the interval $[t, t + \tau)$ is much more than 1, $a_j(\mathbf{X}(t); \boldsymbol{\theta})\tau \gg 1$, $j = 1, \dots, M$. While also restricting τ to be

small enough that the propensity functions $a_j(\mathbf{X}(t); \boldsymbol{\theta})$ do not markedly change in value. This allows us to make a sufficient approximation of our Wiener process by a unit normal random variable [24]. Substituting this result into Equation 8

$$X_i(t + \tau) = X_i(t) + \underbrace{\sum_{j=1}^M \nu_{i,j} a_j(\mathbf{X}(t); \boldsymbol{\theta}) \tau}_{\text{Drift function}} + \underbrace{\sum_{j=1}^M \nu_{i,j} N_j \sqrt{a_j(\mathbf{X}(t); \boldsymbol{\theta}) \tau}}_{\text{Diffusion function}} \quad (9)$$

where N_j are independent standard normal random variables.

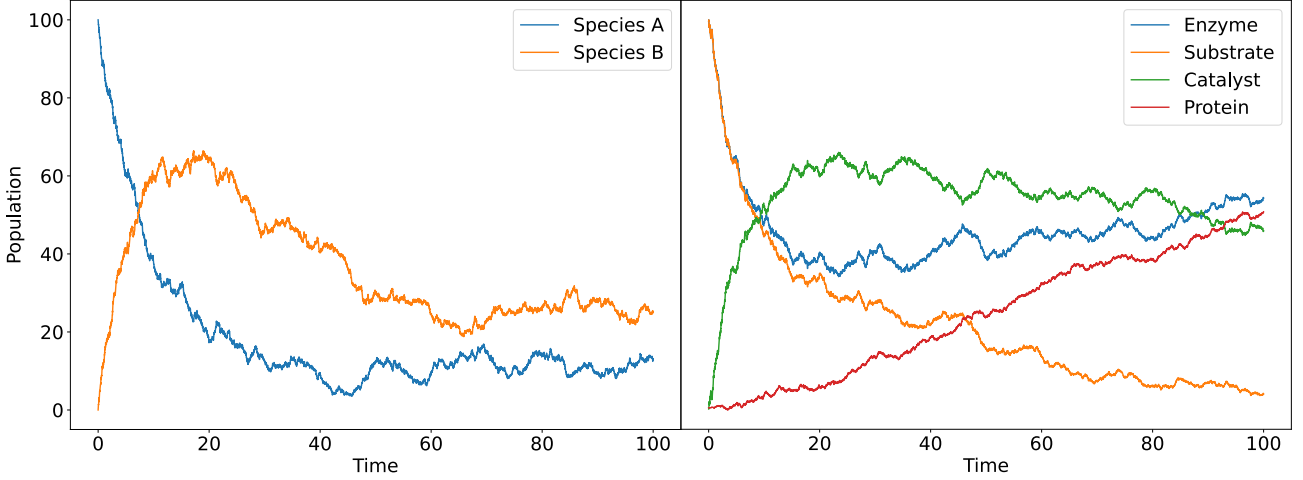


Figure 3: (left) Euler-Maruyama method sample path of the mono-molecular chain model using Julia on CPU. With kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$, initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$. (right) Euler-Maruyama method sample path of the Michaelis-Menten enzyme kinetic chain model using Julia on CPU. With kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$, initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$

3.2 Random Number Generation

It is well recognised that the generation of random numbers for use within the described algorithms is the most significant bottleneck for performance. In stochastic simulation, it is common to use algorithms such as the Mersenne-Twister algorithm due to preferable statistical independence and uniformity [70]. However in low-level parallel computing, such an algorithm causes serialisation since it relies on sequentially dependent state transformations [71–73]. This is especially impactful in our algorithms, since the random numbers are generated in the inner-most loop of the simulation. Low-level parallelism is often done using both multi-threading and single instruction, multiple data (SIMD) operations [74, 75], the PRNG serialisation can destroy the SIMD sequence and cause sequential execution.

Throughout this paper, we utilise standard libraries for random number generator implementations. Specifically, we use the mrg63k3a algorithm in C++ and OpenCL implementations [72], this was not however easily accessible in Julia, as such we utilised the standard linear congruential generator (LCG). We choose these generators as they perform well in parallel computing and are easily accessible within standard libraries. Since we are unable to use a the mrg63k3a algorithm in Julia, we utilise a less time complex generator (LCG) to maintain fairness of comparison since we expect Julia to perform the worst.

3.3 Implementation

3.3.1 The OpenCL Programming Language

OpenCL is an open source programming standard and API for heterogeneous computing. The standard enables the compilation of OpenCL code to target a wide variety of compute devices — such

as AMD, ARM and Intel GPU's — without significant code restructuring. This standard is hence extremely useful when we wish to utilise a variety of devices and accelerators to conduct our computation.

The OpenCL standard defines several key terminologies when referring to heterogeneous and parallel computing environments. The *host* is the compute device that is responsible for queuing kernels and reading/writing information to and from the secondary devices, this can be written in C/C++ directly. The *kernel* is the code which is executed on the secondary devices, written in the OpenCL device language (a C-like language). A *compute device* is the hardware that is being targeted, for example the GPU on a computer. Within the compute device we have *compute units*, these can be thought of most directly as the cores of a CPU / GPU. The final layer is the *processing element* which can be thought of as threads or the vector units of a CPU / GPU [76, 77]. When moving to FPGAs these definitions become more tenuous as the device is reconfigured based on the users compiled device code, however, these are the basic premises.

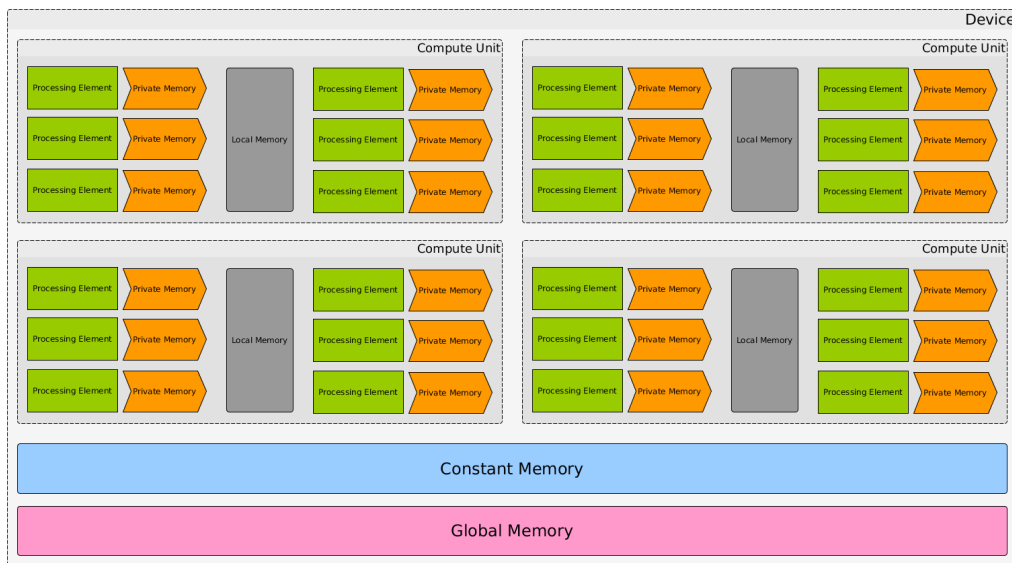


Figure 4: Demonstrative figure of the key terminology used in the OpenCL standard and memory structure.

Since we are focusing on simulation-based methods, it is valuable to explore the memory model of the API. As we are able to take advantage of device side memory management to reduce computational overheads caused by repetitive read-write queries to the host. Within the device there is the *constant* memory type which is shared between all compute units — can be thought of as essentially texture memory on a GPU. This can be used to efficiently store common calculations that are need across all instances, allowing a single compute unit to perform the work and share it across other units. *Private* memory is processing element scope memory and is the quickest for each kernel to access — effectively register level, similar to L1/L2 cache on a CPU. The *global* memory space is memory shared between the host and the kernel, used to read and write buffers between devices, due to this it also has a significant overhead as compared to device-specific memory allocations — device-side main memory [36, 46].

The final layer of terminology comes when we reference the actual code being parsed to the compute device. These are called *kernels*, where a kernel is program which is at its lowest level of parallelism. These kernels run on single processing elements, this is referred to as a *work item* — being individual kernel calls running on the lowest level of a device's processor. A partition of these work items are *work groups*, these are simply a collective of kernels that run on a compute unit. We define these terms, since it is a key consideration within this paper as to effective partitioning of these work items and groups. Listing 1 demonstrates an example of an OpenCL kernel for vector addition

```

1      __kernel void vector_add(__global float* A,
2                               __global float* B,
3                               __global float* C) {

```

```

4         int i = get_global_id(0);
5         C[i] = A[i] + B[i];
6     }

```

Listing 1: Example implementation of simple vector addition kernel in OpenCL. The buffers A, B and C would be written on the host-side C++ code and copied to the device before this execution. The *get_global_id* function returns a unique integer value determining the location of the thread in the total work group. This allows it to write to the output buffer (C) without introducing race conditions

As introduced in [Section 2.3.1](#), the CPU and GPU have varying compute architectures aimed at solving different problems. It is then a question as to what partitioning scheme most effectively takes advantage of these differences to accelerate code performance. For a CPU, at the finest grain we want our code to conform to SIMD principles, it is intuitive to think then that we would partition our compute units to have n processing elements, where n is the SIMD length of the thread. NVIDIA GPU architecture however, would seem to benefit from a scalar multiple of 32 processing units per compute unit, to allow maximum parallelism on each SM, due to the *Warp* grouping structure — essentially just a SIMD register. Within OpenCL, we are able to specify this partitioning through the *NDRange* function, see [Listing 2](#). Here the *global size* is the total number of work items to be computed, and the *local size* is the number of work items to be allocated per work group. An example for the GPU would be, local size = 32 and global size = $32 \times \text{number of simulations}$.

```

1     #define NUM_SIMS 10000
2     #define LOCAL_WORK_SIZE 32
3     #define GLOBAL_WORK_SIZE ((NUM_SIMS + LOCAL_WORK_SIZE - 1) / LOCAL_WORK_SIZE) *
LOCAL_WORK_SIZE
4     size_t local_size = LOCAL_WORK_SIZE;
5     size_t global_size = GLOBAL_WORK_SIZE;
6     cl_int runtime_err = queue.enqueueNDRangeKernel(kernel, cl::NullRange, cl::NDRange(
global_size), cl::NDRange(local_size))

```

Listing 2: Example initialisation code of the partitioning scheme for the target device. OpenCL utilises the `enqueueNDRangeKernel` key word to launch of kernel on the device. By specifying both a local work size and a global size, we dictate a matrix of kernel executions on the GPU.

3.3.2 Approximate stochastic simulation implementation and optimisation

As discussed in [Section 3.3.1](#), in order to implement our stochastic simulation algorithms, we must write both host-side — the controller — and device-side — the worker — code. We chose to use C++ to write the host code. Mostly, this involves setting up the control flow and buffers for the device to begin executing kernels. Within the scope of this paper, we focused mainly on a single kernel running on a single device and subsequently the host code is somewhat trivial. The host code becomes more complex, however, when we implement the MLMC scheme as this involves tracking data across multiple large simulation runs — not implemented in this paper. Meaning, we require much more complex memory management systems and race condition protections. The primary complexity coming from the selection of work group size, demonstrated in [Listing 2](#).

When developing the OpenCL kernels, a key consideration was how to access random numbers during simulation, since OpenCL does not utilise the standard random C/C++ libraries. Initially, we developed a CPU parallelised random number generation sequence which computed the required random numbers for all simulations before passing this buffer to the GPU. The kernel would then use its unique simulation identifier to load random numbers as needed.

```

1     int num_cores = 20;
2     omp_set_num_threads(num_cores);
3     double rand_unifs_h[NUM_RANDS_NEEDED];
4     #pragma omp parallel
5     {
6         #pragma omp for
7         for (size_t i = 0; i < NUM_RANDS_NEEDED; i++){

```

```

8         thread_local static std::uniform_real_distribution<double> unif_dist(0.0,1.0);
9         thread_local static std::random_device rd;
10        thread_local static std::mt19937 prng(rd());
11        rand_unifs[i] = unif_dist(prng);
12    }
13 }

```

Listing 3: Example algorithm to produce the necessary uniform pseudo-random numbers required to compute the tau leaping method. We note the utilisation of the *thread_local* keyword which allows us to create thread-based random number engines. Meaning we greatly reduce the time complexity of the operation.

This however, meant that we were performing the most time complex operation on the CPU still, and thus this did not provide the expected results in terms of computational speed up. To solve this, we instead chose to utilise an OpenCL random number generation library, RandomCL [72], specifically using the mrg63k3a 64-bit multiplicative recursive generator. This allows us to produce uniform random numbers within the OpenCL kernel, which we are then able to transform into the desired random number distribution — see Listing 4 for the Poisson random number generator used in the Tau leaping implementation.

```

1  int sample_poisson(const double lambda, mrg63k3a_state* state){
2      const double p = (double)(mrg63k3a_advance(state)) / (double)MRG63K3A_M1;
3      const double exp_lambda = exp(-lambda);
4      double p_k = exp_lambda;
5      double p_cumulative = p_k;
6      int k = 0;
7      while (p_cumulative < p){
8          k++;
9          p_k *= lambda / k;
10         p_cumulative += p_k;
11     }
12     return k;
13 }

```

Listing 4: Example function that utilises a multiplicative recursive generator to produce Poisson distributed random numbers. Utilised in the Tau leaping method to determine the number of reaction events that have occurred in the timestep τ . This algorithm is written such that it can with no out-of-thread dependencies on the GPU.

We provide the main loop within the Tau leaping kernel in Listing 5 as an example implementation. This is programmed based on the form derived in Equation 6 and utilises the Poisson distributed random number generator described in Listing 4.

```

1  for (size_t t = 1; t < N; t++){
2      for (size_t cur_reaction = 0; cur_reaction < num_reactions; cur_reaction++){
3          // Access the 'function array' and evaluate
4          __private double cur_propensity = propensity_function(cur_reaction, temp_x, k);
5          // Calculate lambda for rand num generations
6          __private double lambda = cur_propensity * tau;
7          // Generate poisson random number
8          __private int reaction_events = sample_poisson(lambda, &state);
9          // Apply the reaction events to local x variable
10         for (size_t species = 0; species < num_species; species++){
11             temp_x[species] += nu[GET_NU_IND(species, cur_reaction, num_species)] *
reaction_events;
12         }
13     }
14     // Write out current timestep populations
15     for (size_t species = 0; species < num_species; species++){
16         x[GET_X_IND(t, N, species, num_species, sim_id)] = temp_x[species];
17     }
18 }

```

Listing 5: Main loop of the Tau leaping OpenCL kernel, noting some variable definitions not included for brevity.

3.4 Comparison Criteria

To demonstrate the differences in performance between the CPU and GPU, we utilise three varying programming languages. Julia is a language that is already widely used by the mathematical modelling tool, which is able to be configured to run in parallel on a CPU. Further to this, we use C++ as a lower level CPU parallelisable coding language, allowing more fine grain optimisations to be had. For GPU programming we then use C++ for the host-side code and OpenCL for device-side kernel programming. We perform two different kernel partitions for the GPU in OpenCL, one with a local work group size of 32 and the other with a size of 1024. We choose these two sizes as through experimental testing we did not see meaningful differences between the performance of 32 and 64, 128, and between 1024 and 256, 512. We did however, see a noticeable performance decrease when using the larger partitions. Through this we aim to demonstrate the importance of partition selection, an issue which becomes more prevalent when moving to FPGA implementations.

To generate fair comparisons between the three implementations, we conduct an average runtime experiment over 100 iterations. This is done by generating and writing the output to local memory of 10,000 simulations per run. We conduct testing this way to ensure that the CPU and GPU are put under full load for a meaningful duration. This is important for making the power consumption calculation assumptions as we assume maximum draw during runtime to calculate total power used. We formalise our power consumption equation to be

$$E = P_{\text{inst}} \times h_{\text{avg-runtime}}, \quad (10)$$

where E is the energy in Watt-hours, P_{inst} is the instantaneous power draw at maximum load in Watts, $h_{\text{avg-runtime}}$ is the average runtime of 10,000 simulations. We utilise the commands listed in [Listing 6](#) and [Listing 7](#) to determine the instantaneous power of each component during operation. Importantly for the inference problem, we require that the stationary distribution of the populations

```
1 sudo watch -n 0.1 'nvidia-smi --query-gpu=power.draw --format=csv,noheader,nounits'
```

Listing 6: Linux terminal command used to find the instantaneous power draw in Watts of the GPU during computation

```
1 sudo turbostat --show PkgWatt --interval 0.1
```

Listing 7: Linux terminal command to query the instantaneous power draw in Watts of the CPU during computation

remain the same between programming methods. We perform a mean and standard deviation analysis on each method at the arbitrary simulation end point $T = 100$ to ensure that we have consistent results between each programming model.

4 Results

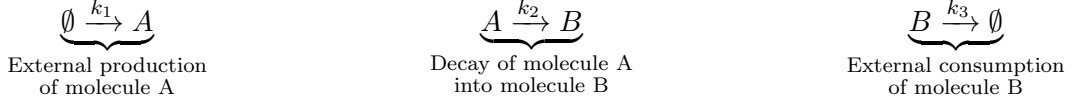
4.1 Example Biochemical Networks

To demonstrate the methods described within this paper, we choose two biochemical reaction networks to be used as representative examples.

4.1.1 Mono-molecular Chains

The mono-molecular chain (MMC) reaction network describes the external production of a molecule A, the decay from A into molecule B, and the external consumption of B [9]. This provides a foundational

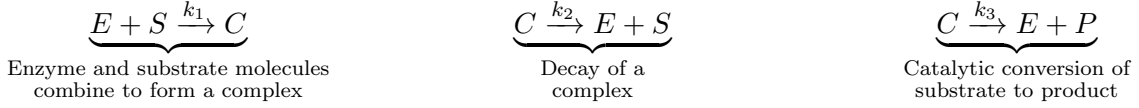
example, and is also able to be solved analytically using the chemical master equation (CME) — note that we do not perform the analytic solution in this paper [24, 78].



Here, k_1 , k_2 and k_3 are the kinetic rate parameters of the system. With the copy number at each state being represented by $\mathbf{x}(t) = [A(t), B(t)]^\top$. We describe the propensity functions and stoichiometric matrix to be $\mathbf{a}(\mathbf{x}(t)) = [k_1, k_2 A(t), k_3 B(t)]$, and $\boldsymbol{\nu} = [\boldsymbol{\nu}_1 | \boldsymbol{\nu}_2 | \boldsymbol{\nu}_3]$ with $\boldsymbol{\nu}_1 = [1, 0]^\top$, $\boldsymbol{\nu}_2 = [-1, 1]^\top$ and $\boldsymbol{\nu}_3 = [0, -1]^\top$.

4.1.2 Michaelis-Menton Enzyme Kinetics

The Michaelis-Menton enzyme kinetic (EK) model describes the catalytic conversion of a substrate, S, into a product, P, via an enzymatic reaction involving enzyme, E [9, 79]. Unlike the MMC network, the CME for the EK reaction network is analytically intractable, and as such gives us a better idea of how the proposed methods will perform for more complex systems. It is also important as it forms a building block for many other larger biological models [80–83]



We have the state vector of the system $\mathbf{x}(t) = [E(t), S(t), C(t), P(t)]^\top$ and again k_1 , k_2 and k_3 are the kinetic rate parameters. With propensity function and stoichiometric matrix described by $\mathbf{a}(\mathbf{x}(t)) = [k_1 E(t) S(t), k_2 C(t)]$ and $\boldsymbol{\nu} = [\boldsymbol{\nu}_1 | \boldsymbol{\nu}_2 | \boldsymbol{\nu}_3]$ with $\boldsymbol{\nu}_1 = [-1, -1, 1, 0]^\top$, $\boldsymbol{\nu}_2 = [1, 1, -1, 0]^\top$ and $\boldsymbol{\nu}_3 = [1, 0, -1, 1]^\top$.

4.2 Accuracy of Methods

Statistical inference relies heavily on the ability to produce many expensive, high-fidelity simulations to perform accurate estimation. It is, therefore, important that our methods must both perform better *and* maintain the quality of simulation data to be later used for inference. The two key pieces of information required are accurate simulate paths of the systems state vectors over a given time interval, and the unconditional state distribution at a given time.

4.2.1 Path Accuracy

Figure 5 and Figure 6 clearly demonstrate the ability of the proposed kernel-based GPU methods to follow the time dependent path of the system over some interval $t = [0, T]$. Importantly, the mean and standard deviation of the paths are visually extremely similar, which is critical for correct parameter inference. From Figure 5 we see that there are small discrepancies in the lower bound of the standard deviation of the molecule paths. This could be attributed to the number of simulations used in the calculation since we utilise 1,000 simulations within our example model. However, we would expect this error to go to zero as the the number of simulations $R \rightarrow \infty$. We generate comparison models for all three simulation methods, however, we only show the Tau leaping method in the below figures for brevity.

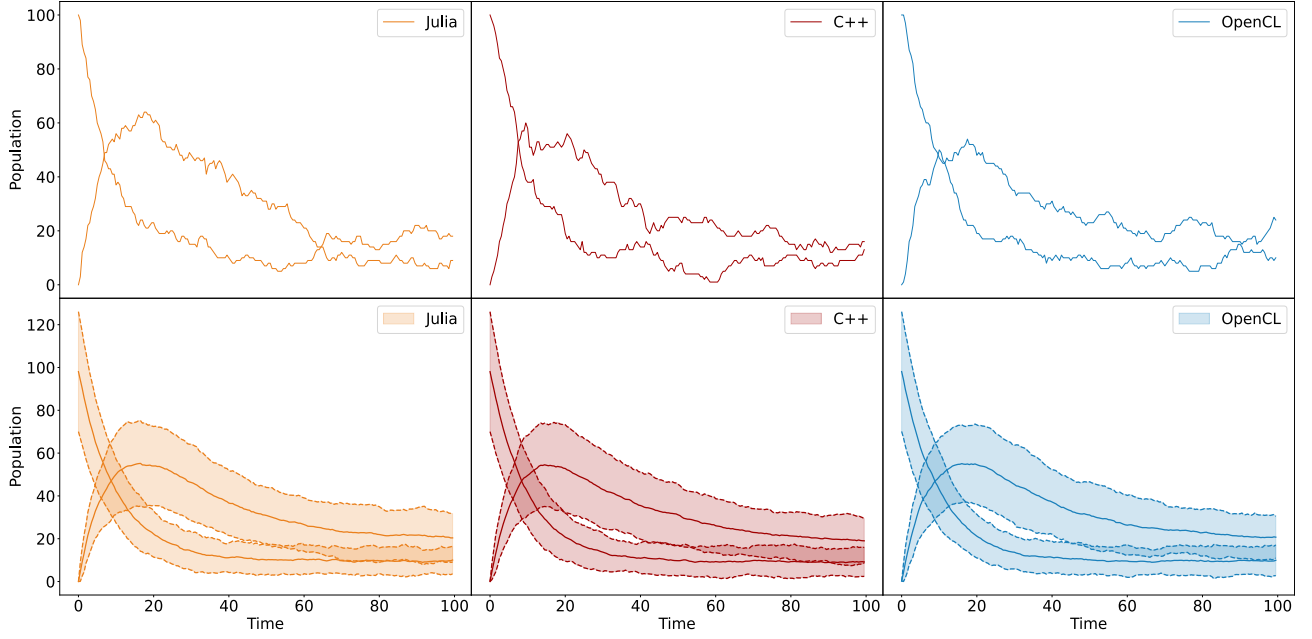


Figure 5: (top row) Tau-leaping sample paths for the mono-molecular chain model using the three programming languages: Julia on CPU (yellow); C++ on CPU (red); and OpenCL on GPU (blue). With kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$ with initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$ and timestep $\tau = 0.1$. (bottom row) Mean (solid line) and two standard deviations (dashed line) of the copy numbers of A and B using the three methods over the same model and parameter set. Demonstrating the accuracy of model paths using differing computation techniques over 1000 simulations.

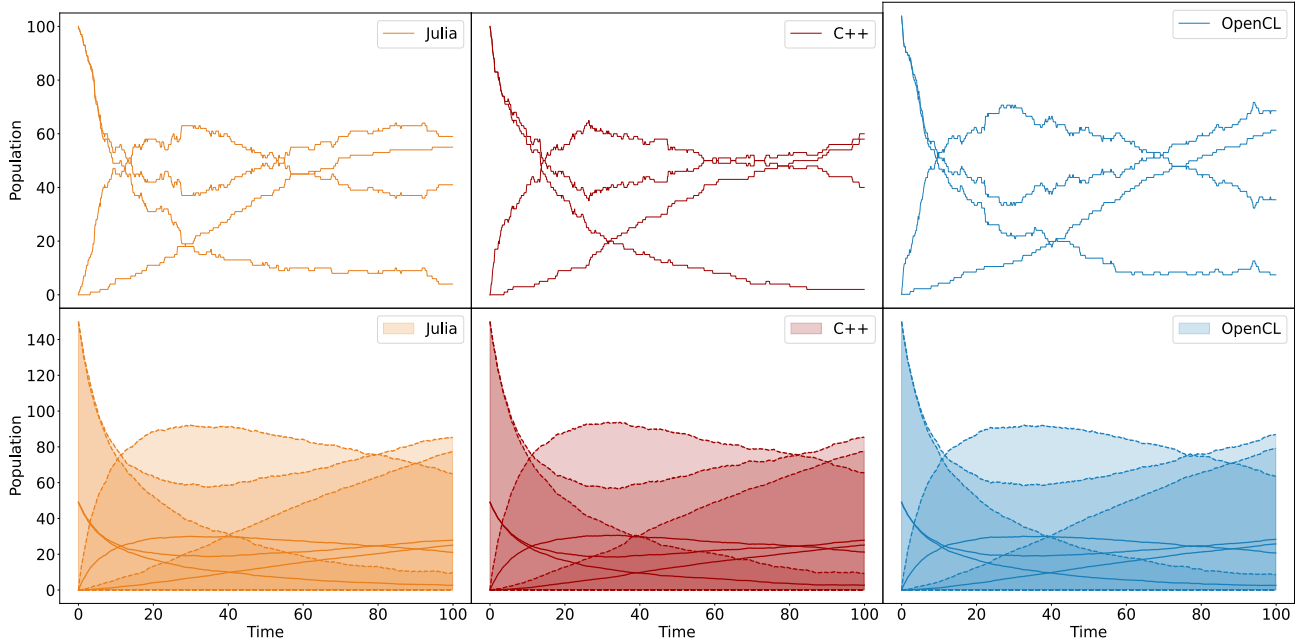


Figure 6: (top row) Tau-leaping sample paths for the Michaelis-Menten enzyme kinetic model using the three programming languages: Julia on CPU (yellow); C++ on CPU (red); and OpenCL on GPU (blue). With kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$ with initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$ and timestep $\tau = 0.1$. (bottom row) Mean (solid line) and two standard deviations (dashed line) of the copy numbers of E, S, C and P using the three methods over the same model and parameter set. Demonstrating the accuracy of model paths using differing computation techniques over 1000 simulations.

4.2.2 State Distribution Accuracy

To demonstrate the accuracy of the state distribution of the methods at any final time T , we find the mean and standard deviation of the points over all simulations at T . We demonstrate these

distributions only using the Tau leaping method in the figures for conciseness, however, we perform the analysis on all methods. By visual inspection of [Figure 7](#) and [Figure 8](#) we can see that our implementation has retained much of the information. However, there are small discrepancies in both the C++ and OpenCL implementations. As discussed in [Section 4.2.1](#) we would expect this deviation to reduce to 0 in the limit at $R \rightarrow \infty$. In the Michaelis-Menton system, we see a more prominent

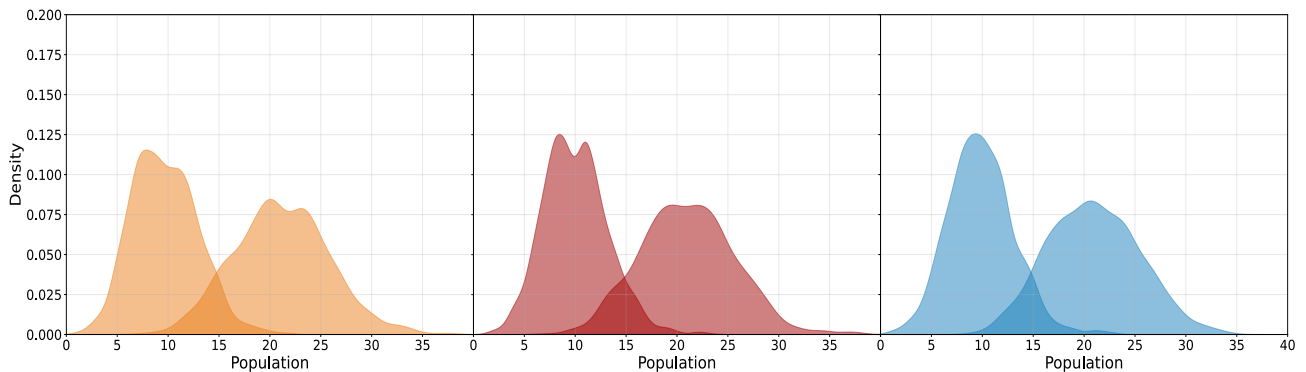


Figure 7: Stationary distribution of the copy numbers of molecules A and B using the Tau leaping method for the mono-molecular chain model using the three programming languages: Julia on CPU (yellow); C++ on CPU (red); and OpenCL on GPU (blue). With kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$ with initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 1000$. Demonstrating the agreement in end time distributions of the different computation methods.

difference in the final state distribution of the molecules. This may be exacerbated by the addition of more complex path ways that the molecules follow as compared to the MMC model. As can be seen in [Figure 6](#) the path of the system appears to track well between all methods. [Table 1](#) and [Table 2](#) numerically demonstrate the agreement between all three method implementations.

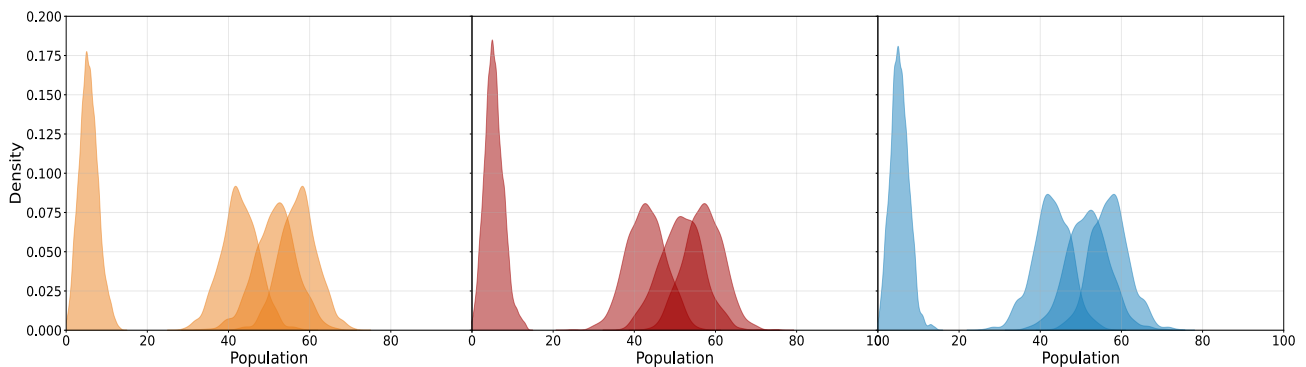


Figure 8: Stationary distribution of the copy numbers of molecules E, S, C, and P using the Tau leaping method for the mono-molecular chain model using the three programming languages (Julia, C++ and OpenCL respectively) with kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$ with initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 1000$. Demonstrating the agreement in end time distributions of the different computation methods.

Table 1: Mono-molecular chain mean and standard deviation result table for the three different methods implemented. Produced using kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$ with initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 10000$. Demonstrating the stationary distribution of the copy numbers of each species at an arbitrary end time T .

		Tau Leap			Gillespie Direct			Euler-Maruyama		
		Julia	C++	OpenCL	Julia	C++	OpenCL	Julia	C++	OpenCL
Species A	Mean	9.078	9.137	9.019	10.085	9.328	10.039	10.044	10.016	10.015
	Std	3.336	3.370	3.589	3.171	3.028	3.148	3.164	3.246	3.201
Species B	Mean	20.686	19.019	20.549	21.038	19.272	21.048	21.056	20.936	21.087
	Std	5.912	5.171	4.879	4.575	4.448	4.611	4.641	4.710	4.664

Table 2: Michaelis-Menton mean and standard deviation result table for the three different methods implemented. Produced using kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$ with initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 10000$. Demonstrating the stationary distribution of the copy numbers of each species at an arbitrary end time T .

		Tau Leap			Gillespie Direct			Euler-Maruyama		
		Julia	C++	OpenCL	Julia	C++	OpenCL	Julia	C++	OpenCL
Enzyme	Mean	28.039	27.824	28.333	57.674	57.470	57.522	57.176	57.316	57.211
	Std	28.320	28.867	29.301	4.742	4.711	4.679	4.712	4.826	4.695
Substrate	Mean	2.431	2.686	2.569	5.385	5.205	5.410	5.476	5.465	5.542
	Std	2.803	3.265	3.041	2.302	2.744	2.301	2.309	2.305	2.284
Catalyst	Mean	20.314	21.196	20.686	42.326	42.530	42.479	42.824	42.684	42.789
	Std	21.602	22.106	21.469	4.742	4.711	4.679	4.712	4.826	4.695
Protein	Mean	25.529	25.137	25.765	52.289	52.266	52.111	51.700	52.133	51.669
	Std	25.882	26.244	26.721	5.251	5.089	5.163	5.222	5.273	5.196

4.3 Computational Performance

To properly unpack our results with the goal of less power consumption per calculation, we must consider if an algorithm is not only faster to compute, but also the power cost associated with the computation. This is a non-trivial result as even though we expect the GPU to perform better in highly parallel operations, such as the simulations being tested, it must do so while also remaining power efficient. All runtime and power calculations done in this section were computed using an Intel Ultra 7 255HX CPU and a Nvidia GeForce RTX 5060 Laptop GPU.

To explore the difference in GPU partitioning we also demonstrate the difference in performance between a local work group size of 32 and 1024. We use scalar multiples of 32 since the GPU being used for benchmarking has a 32 thread Warp. It is expected that there will be a difference in performance between the two local work group sizes as we are changing the number of simulations run on each streaming multiprocessor (SM) — essentially a core. Each SM — on the Nvidia Geforce RTX 5060 — contains 48 Warps, however, it only has four Warp schedulers, meaning that any partitioning scheme that has greater than 128 threads will queue the oncoming processes until the previous simulations have completed.

4.3.1 Runtime Efficiency

We benchmark the average runtime of our proposed methods to demonstrate the substantial benefits gained through using a heterogeneous computing framework. For the Tau-leap and Euler-Maruyama methods we display the runtimes for a timestep $\tau = 0.1$ in [Table 3](#) and [Table 4](#), and demonstrate the change in runtime as we reduce τ in [Figure 9](#) and [Figure 10](#).

We see that in [Table 3](#) and [Table 4](#) the Tau leaping and Euler-Maruyama schemes experience large runtime improvements as compared to the Julia code. Conversely, the OpenCL implementation of the

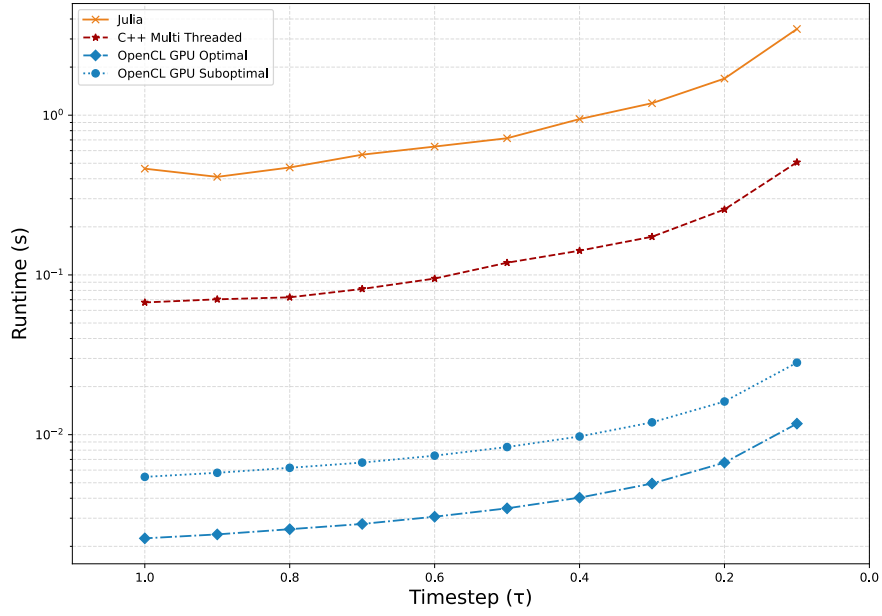


Figure 9: Comparison of the run time of tau leaping method using the mono-molecular chain model with kernels written in Julia (yellow solid line with cross markers), C++ (red dashed line with star markers), and OpenCL (local work group size of 32 in blue dash-dotted line and diamond markers, and 1028 in blue dotted line with circle markers). Computed using kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$ with initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 10000$. Demonstrating the advantages of kernel based GPU accelerated simulation methods.

Gillespie direct method has a much smaller computational speed up. This is an expected result, as the two approximate simulation algorithms have a set number of steps in each simulation. This makes the SIMD operation flow much more predictable and easier to exploit in programming methods. The exact method however, has a random number of timesteps, meaning we lose the potential for SIMD among the simulations in each Warp.

Table 3: Comparison between the run times of each stochastic simulation method using the mono-molecular chain model. Computed using kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$ with initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 10000$. Demonstrating the advantages of kernel based GPU accelerated simulation methods. Noting that the OpenCL code was run using a local work group size of both 32 and 1024. The speed up is calculated by comparing the Julia and OpenCL [32] runtimes.

	Julia	C++	OpenCL [32]	OpenCL [1024]	Speedup (%)
Tau Leaping	3.463	0.507	0.012	0.028	295.4
Gillespie Direct	0.228	0.513	0.004	0.009	59.5
Euler-Maruyama	9.062	0.873	0.026	0.031	353.9

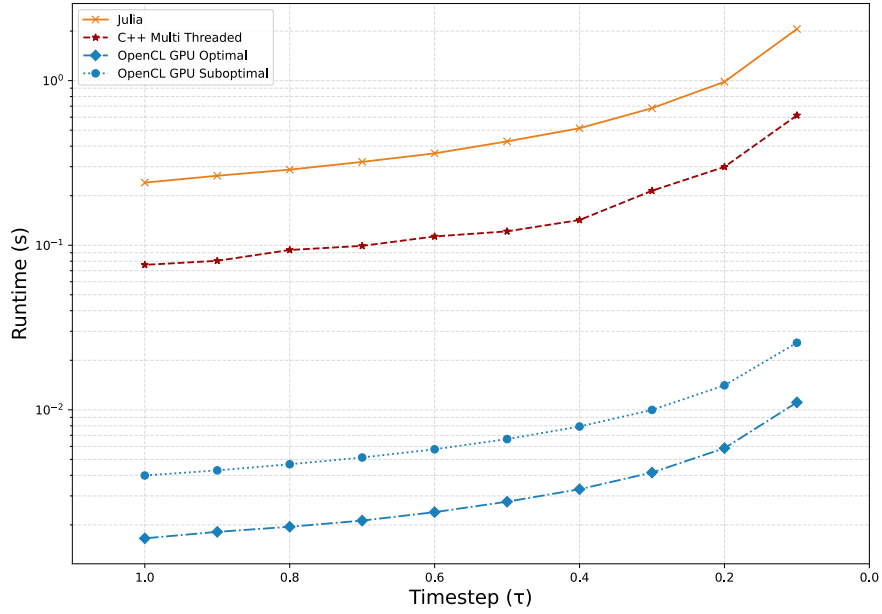


Figure 10: Comparison of the run time of tau leaping method using the Michaelis-Menton model with kernels written in Julia (yellow solid line with cross markers), C++ (red dashed line with star markers), and OpenCL (local work group size of 32 in blue dash-dotted line and diamond markers, and 1028 in blue dotted line with circle markers). Computed using kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$ with initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 10000$. Demonstrating the advantages of kernel based GPU accelerated simulation methods.

Table 4: Comparison between the run times of each stochastic simulation method using the Michaelis-Menton model. Computed using kinetic rate parameters $k_1 = 0.001$, $k_2 = 0.005$ and $k_3 = 0.01$ with initial conditions $E(0) = 100$, $S(0) = 100$, $C(0) = 0$, $P(0) = 100$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 10000$. Demonstrating the advantages of kernel based GPU accelerated simulation methods. Noting that the OpenCL code was run using a local work group size of both 32 and 1024. The speed up is calculated by comparing the Julia and OpenCL [32] runtimes.

	Julia	C++	OpenCL [32]	OpenCL [1024]	Speedup (%)
Tau Leaping	2.059	0.615	0.011	0.026	185.5
Gillespie Direct	0.231	0.666	0.010	0.012	23.8
Euler-Maruyama	9.115	0.901	0.030	0.036	304.7

4.3.2 Power Efficiency

To measure the power usage of our different devices during simulation load, we utilise Linux terminal commands, defined in Listing 6 and Listing 7, to query power draw during execution. To calibrate the results, a sufficiently large simulation was used to ensure maximum device utilisation such that the power levelled at a consistent wattage — approximately 5 seconds of continuous load. The average power draw was then recorded and used for calculations. It was found that during peak load, the CPU drew approximately 30W, and the GPU drew approximately 50W. We then computed the energy $E = P_{\text{load}} \times t_{\text{sim}}$, where P_{load} is the average instantaneous power draw during load, and t_{sim} is the

average simulation time of 10,000 simulations.

As shown in Figure 11 and the proposed OpenCL kernel for simulation performs at a power efficiency of nearly two orders of magnitude more than the Julia implementation — similar results were achieved in the EK model but not shown here for brevity. This result is to be expected by the nature of our calculation, since the kernel achieves a nearly 300 time computational speed up over the Julia code. This result however, should be taken lightly since the command dictated in Listing 6 does not account for the power utilised to cool the GPU. It is also unrealistic to assume the conditions experienced by a laptop GPU would be similar to that of a data centre in terms of cooling expenditure. Further work should look to quantify this result using actual HPC equipment.

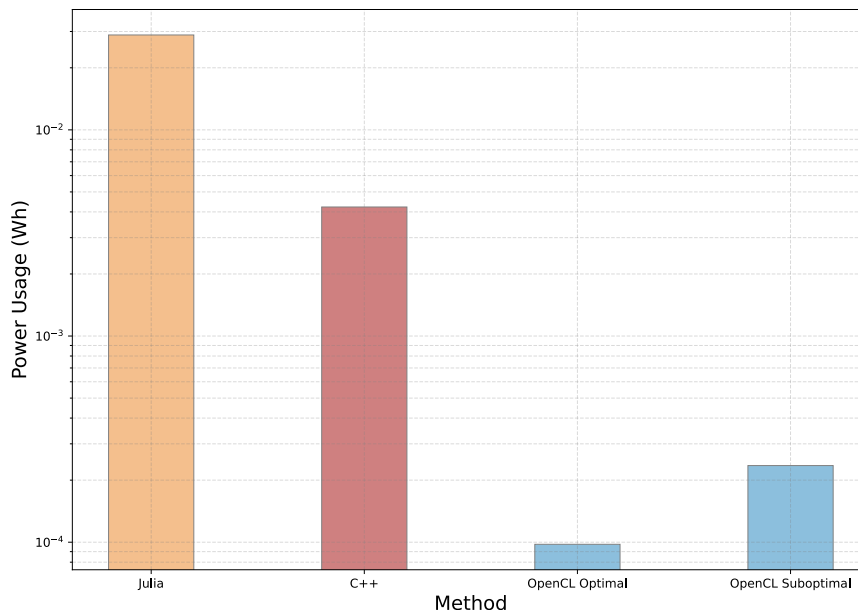


Figure 11: Energy utilisation of the Tau leaping method using the mono-molecular chain model. Power usage calculated by $E = P \times t$, where E is the total power usage, P is the instantaneous power usage — 30W for CPU and 50W for GPU — and t is the average runtime of 10,000 simulations. Simulation parameters being; kinetic rate parameters $k_1 = 1.0$, $k_2 = 0.1$ and $k_3 = 0.05$ with initial conditions $A(0) = 100$, $B(0) = 0$, final time $T = 100$, timestep $\tau = 0.1$ and $R = 10000$.

5 Discussion

The proposed methods in this paper commonly used for computational inference analyses, Tau leaping and Euler-Maruyama, were able to achieve approximately 300 times speed up in computational performance while consuming approximately 170 times less power. Moreover, we do not see any perceptible decrease in simulation quality in terms of statistical quantities. These power efficiencies do not fully capture the real-world consumption of large GPU dense data centres though. We expect that substantially more energy would be consumed in the process of actively cooling these tightly packed server racks. However, the scale of the improvement leads us to believe that there is still major room to be had from a heterogeneous approach to the inference problem.

To explore the benefits that these algorithms may produce in the inference problem for biochemical reaction networks, we will target future work on a heterogeneous implementation of the mixed precision MLMC scheme proposed by Haas [38]. The goal of the work is to demonstrate the potential benefits that may be exploited by the unique ability of FPGAs to utilise custom floating point precision data. In doing so, we will use the portable OpenCL frameworks developed within this paper, and expand them to optimally target FPGA architectures.

A key limitation of the work we have produced in this paper is the hardware used when generating results. Since the methods proposed are designed to be used in HPC, the power efficiencies determined do not encompass the full range of complications faced when computing on this large scale. Active cooling solutions for GPU dense data centres accounts for a substantial percentage of their overall power consumption.

Our findings suggest that there are significant improvements that can be made to the efficiency of simulating complex biochemical reaction networks. It is therefore important that we ensure computational methods are able to scale to meet these challenges. Since these complex models will likely be required in the next generation of whole cell modelling and as such it is critical that we are able to reduce wall-time while simultaneously conducting GHG emission conscious computing.

5.1 Project Timeline

Throughout this paper, we have explored the benefits of a CPU-GPU heterogeneous computing environment to accelerate simulation methods in biochemical reaction networks. This is an important step in the direction of mixed precision computing for simulation based inference. In future work we aim to modify our developed methods to utilise varying precision — half, single, and double. This is an important step in the implementation of the proposed mixed precision MLMLC scheme proposed by Haas and Giles [38]. Once this has been developed we then aim to extend the code to be able to target FPGA devices and utilise custom floating point data structures. This will allow us to investigate the practical impact of the proposed method and compare against theoretical findings.

Task	MXB371												
	1	2	3	4	5	6	7	8	9	10	11	12	13
Investigate current methods for simulation based inference in heterogeneous computing													
Implement Tau leaping method													
Implement Gillespie direct method													
Implement Euler-Maruyama method													
Perform validation testing on methods													
Conduct performance evaluations on algorithms													
Validate and finalise formatting of results													
Write report													

Task	MXB372												
	1	2	3	4	5	6	7	8	9	10	11	12	13
Review and understand the mathematical procedure outlined in Haas and Giles paper													
Implement Tau leaping method using FPGA													
Simulate mixed precision and custom floating point operation on Vivado Suite													
Implement MLMC scheme													
Test and validate MLMC scheme using FPGA kernels													
Conduct performance evaluations on algorithms													
Validate and finalise formatting of results													
Write report													

References

- [1] J. L. Abkowitz, S. N. Catlin, and P. Gutter, “Evidence that hematopoiesis may be a stochastic process in vivo,” *Nature Medicine*, vol. 2, no. 2, pp. 190–197, Feb. 1996.
- [2] A. Arkin, J. Ross, and H. H. McAdams, “Stochastic kinetic analysis of developmental pathway bifurcation in phage lambda-infected *Escherichia coli* cells,” *Genetics*, vol. 149, no. 4, pp. 1633–1648, Aug. 1998.
- [3] M. Kærn, T. C. Elston, W. J. Blake, and J. J. Collins, “Stochasticity in gene expression: From theories to phenotypes,” *Nature Reviews Genetics*, vol. 6, no. 6, pp. 451–464, Jun. 2005.
- [4] D. J. Wilkinson, “Stochastic modelling for quantitative description of heterogeneous biological systems,” *Nature Reviews Genetics*, vol. 10, no. 2, pp. 122–133, Feb. 2009.
- [5] P. Guptasarma, “Does replication-induced transcription regulate synthesis of the myriad low copy number proteins of *Escherichia coli*?” *BioEssays*, vol. 17, no. 11, pp. 987–997, 1995.
- [6] M. B. Elowitz, A. J. Levine, E. D. Siggia, and P. S. Swain, “Stochastic Gene Expression in a Single Cell,” *Science*, vol. 297, no. 5584, pp. 1183–1186, Aug. 2002.
- [7] A. Raj and A. van Oudenaarden, “Nature, Nurture, or Chance: Stochastic Gene Expression and Its Consequences,” *Cell*, vol. 135, no. 2, pp. 216–226, Oct. 2008.
- [8] D. J. Higham, “Modeling and Simulating Chemical Reactions,” *SIAM Review*, vol. 50, no. 2, pp. 347–368, Jan. 2008.
- [9] D. J. Warne, R. E. Baker, and M. J. Simpson, “Simulation and inference algorithms for stochastic biochemical reaction networks: From basic concepts to state-of-the-art,” *Journal of The Royal Society Interface*, vol. 16, no. 151, p. 20180943, Feb. 2019.
- [10] L. Lannelongue *et al.*, “GREENER principles for environmentally sustainable computational science,” *Nature Computational Science*, vol. 3, no. 6, pp. 514–521, Jun. 2023.
- [11] “The carbon footprint of computational research,” *Nature Computational Science*, vol. 3, no. 8, pp. 659–659, Aug. 2023.
- [12] L. Lannelongue and M. Inouye, “Carbon footprint estimation for computational research,” *Nature Reviews Methods Primers*, vol. 3, Feb. 2023.
- [13] R. Brightwell, B. W. Barrett, K. S. Hemmert, and K. D. Underwood, “Challenges for High-Performance Networking for Exascale Computing,” in *2010 Proceedings of 19th International Conference on Computer Communications and Networks*, Zurich, Switzerland: IEEE, Aug. 2010, pp. 1–6.
- [14] G. Rostirolla, R. da Rosa Righi, V. F. Rodrigues, P. Velho, and E. L. Padoin, “GreenHPC: A novel framework to measure energy consumption on HPC applications,” in *2015 Sustainable Internet and ICT for Sustainability (SustainIT)*, Apr. 2015, pp. 1–8.
- [15] D. J. Warne, R. Hayward, N. Kelson, J. Banks, and L. Mejias, “Pulse-coupled neural network performance for real-time identification of vegetation during forced landing,” *ANZIAM Journal*, vol. 54, p. 1, Mar. 2014.
- [16] D. J. Warne, N. A. Kelson, and R. F. Hayward, “Comparison of High Level FPGA Hardware Design for Solving Tri-diagonal Linear Systems,” *Procedia Computer Science*, vol. 29, pp. 95–101, 2014.
- [17] M. Sunnåker, A. G. Busetto, E. Numminen, J. Corander, M. Foll, and C. Dessimoz, “Approximate Bayesian Computation,” *PLOS Computational Biology*, vol. 9, no. 1, e1002803, Jan. 2013.
- [18] B. T. Bajar, E. S. Wang, S. Zhang, M. Z. Lin, and J. Chu, “A Guide to Fluorescent Protein FRET Pairs,” *Sensors*, vol. 16, no. 9, p. 1488, Sep. 2016.
- [19] B. Finkenstädt *et al.*, “Reconstruction of transcriptional dynamics from gene reporter data using differential equations,” *Bioinformatics*, vol. 24, no. 24, pp. 2901–2907, Dec. 2008.

- [20] J. Liepe, S. Filippi, M. Komorowski, and M. P. H. Stumpf, “Maximizing the Information Content of Experiments in Systems Biology,” *PLOS Computational Biology*, vol. 9, no. 1, e1002888, Jan. 2013.
- [21] A. P. Browning, S. W. McCue, and M. J. Simpson, “A Bayesian Computational Approach to Explore the Optimal Duration of a Cell Proliferation Assay,” *Bulletin of Mathematical Biology*, vol. 79, no. 8, pp. 1888–1906, Aug. 2017.
- [22] A. M. Ellison, “Bayesian inference in ecology,” *Ecology Letters*, vol. 7, no. 6, pp. 509–520, 2004.
- [23] D. T. Gillespie, “Approximate accelerated stochastic simulation of chemically reacting systems,” *The Journal of Chemical Physics*, vol. 115, no. 4, pp. 1716–1733, Jul. 2001.
- [24] D. T. Gillespie, “The chemical Langevin equation,” *The Journal of Chemical Physics*, vol. 113, no. 1, pp. 297–306, Jul. 2000.
- [25] D. T. Gillespie, “Exact stochastic simulation of coupled chemical reactions,” *The Journal of Physical Chemistry*, vol. 81, no. 25, pp. 2340–2361, Dec. 1977.
- [26] J. K. Pritchard, M. T. Seielstad, A. Perez-Lezaun, and M. W. Feldman, “Population growth of human Y chromosomes: A study of Y chromosome microsatellites,” *Molecular Biology and Evolution*, vol. 16, no. 12, pp. 1791–1798, Dec. 1999.
- [27] K. Cranmer, J. Brehmer, and G. Louppe, “The frontier of simulation-based inference,” *Proceedings of the National Academy of Sciences*, vol. 117, no. 48, pp. 30 055–30 062, Dec. 2020.
- [28] G. Papamakarios, D. Sterratt, and I. Murray, “Sequential Neural Likelihood: Fast Likelihood-free Inference with Autoregressive Flows,” in *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, PMLR, Apr. 2019, pp. 837–848.
- [29] L. F. Price, C. C. Drovandi, A. Lee, and D. J. Nott, “Bayesian Synthetic Likelihood,” *Journal of Computational and Graphical Statistics*, vol. 27, no. 1, pp. 1–11, Jan. 2018.
- [30] S. N. Wood, “Statistical inference for noisy nonlinear ecological dynamic systems,” *Nature*, vol. 466, no. 7310, pp. 1102–1104, Aug. 2010.
- [31] C. Andrieu and G. O. Roberts, “The pseudo-marginal approach for efficient Monte Carlo computations,” *The Annals of Statistics*, vol. 37, no. 2, pp. 697–725, Apr. 2009.
- [32] Z. Ding *et al.*, “Tracking the carbon footprint of global generative artificial intelligence,” *The Innovation*, vol. 6, no. 5, p. 100 866, May 2025.
- [33] T. Aslan, P. Holzapfel, L. Stobbe, A. Grimm, N. F. Nissen, and M. Finkbeiner, “Toward climate neutral data centers: Greenhouse gas inventory, scenarios, and strategies,” *iScience*, vol. 28, no. 1, p. 111 637, Jan. 2025.
- [34] A. Banerjee, T. Mukherjee, G. Varsamopoulos, and S. K. S. Gupta, “Cooling-aware and thermal-aware workload placement for green HPC data centers,” in *International Conference on Green Computing*, Aug. 2010, pp. 245–256.
- [35] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?” In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, Virtual Event Canada: ACM, Mar. 2021, pp. 610–623.
- [36] H. J. Macintosh, J. E. Banks, and N. A. Kelson, “Implementing and Evaluating an Heterogeneous, Scalable, Tridiagonal Linear System Solver with OpenCL to Target FPGAs, GPUs, and CPUs,” *International Journal of Reconfigurable Computing*, vol. 2019, pp. 1–13, Oct. 2019.
- [37] *Comparing Energy Efficiency of CPU, GPU and FPGA Implementations for Vision Kernels*, https://www.researchgate.net/publication/333199244_Comparing_Energy_Efficiency_of_CPU_GPU_and_FPGA_Implementations_for_Vision_Kernels
- [38] I.-B. Haas and M. B. Giles, *A nested MLMC framework for efficient simulations on FPGAs*, Feb. 2025.
- [39] D. J. Warne, N. A. Kelson, and R. F. Hayward, “Solving Tri-Diagonal Linear Systems using Field Programmable Gate Arrays,”

- [40] D. Kaeli, P. Mistry, D. Schaa, and D. P. Zhang, “Chapter 1 - Introduction,” in *Heterogeneous Computing with OpenCL 2.0*, D. Kaeli, P. Mistry, D. Schaa, and D. P. Zhang, Eds., Boston: Morgan Kaufmann, Jan. 2015, pp. 1–14.
- [41] *June 2025 | TOP500*, <https://top500.org/lists/top500/2025/06/>.
- [42] *June 2025 | TOP500*, <https://top500.org/lists/green500/2025/06/>.
- [43] S. Mittal and J. Vetter, “A survey of CPU-GPU heterogeneous computing techniques,” *ACM Computing Surveys*, vol. 47, Jul. 2015.
- [44] I. Paul, V. Ravi, S. Manne, M. Arora, and S. Yalamanchili, “Coordinated energy management in heterogeneous processors,” in *SC '13: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, Nov. 2013, pp. 1–12.
- [45] J. M. P. Cardoso, J. G. F. Coutinho, and P. C. Diniz, “Chapter 2 - High-performance embedded computing,” in *Embedded Computing for High Performance*, J. M. P. Cardoso, J. G. F. Coutinho, and P. C. Diniz, Eds., Boston: Morgan Kaufmann, Jan. 2017, pp. 17–56.
- [46] H. J. Macintosh, D. J. Warne, N. A. Kelson, J. E. Banks, and T. W. Farrell, “Implementation of parallel tridiagonal solvers for a heterogeneous computing environment,” *ANZIAM Journal*, vol. 56, p. 446, Feb. 2016.
- [47] J. Banks *et al.*, “CALCULATION OF ENGINE PARAMETERS USING RECONFIGURABLE HARDWARE,” *International Journal of Computational Methods*, vol. 11, no. supp01, p. 1 344 011, Nov. 2014.
- [48] S. Kulkarni, A. Tsyplikhin, M. M. Krell, and C. Moritz, *Accelerating Simulation-based Inference with Emerging AI Hardware*. Dec. 2020, p. 132.
- [49] Z. Jia, B. Tillman, M. Maggioni, and D. P. Scarpazza, *Dissecting the Graphcore IPU Architecture via Microbenchmarking*, Dec. 2019.
- [50] A. Fitzgibbon and S. Felix, *On Stochastic Rounding with Few Random Bits*, Apr. 2025.
- [51] P.-S. V. Sun *et al.*, *Intelligence Processing Units Accelerate Neuromorphic Learning*, Nov. 2022.
- [52] “Mixed-Precision Arithmetic for AI: A Hardware Perspective,”
- [53] M. Klaisoongnoen, N. Brown, and O. T. Brown, “Low-power option Greeks: Efficiency-driven market risk analysis using FPGAs,” in *International Symposium on Highly-Efficient Accelerators and Reconfigurable Technologies*, Jun. 2022, pp. 95–101.
- [54] A. O Mahony, B. Hanzon, and E. Popovici, “The Role of FPGAs in Modern Option Pricing Techniques: A Survey,” *Electronics*, vol. 13, no. 16, p. 3186, Jan. 2024.
- [55] L. Ma, F. B. Muslim, and L. Lavagno, “High Performance and Low Power Monte Carlo Methods to Option Pricing Models via High Level Design and Synthesis,” in *2016 European Modelling Symposium (EMS)*, Nov. 2016, pp. 157–162.
- [56] X. Tian and K. Benkrid, “Implementation of the Longstaff and Schwartz American Option Pricing Model on FPGA,” *J. Signal Process. Syst.*, vol. 67, no. 1, pp. 79–91, Apr. 2012.
- [57] M. B. Giles, “Multilevel Monte Carlo Path Simulation,” *Operations Research*, vol. 56, no. 3, pp. 607–617, Jun. 2008.
- [58] D. F. Anderson and D. J. Higham, “Multilevel Monte Carlo for Continuous Time Markov Chains, with Applications in Biochemical Kinetics,” *Multiscale Modeling & Simulation*, vol. 10, no. 1, pp. 146–179, Jan. 2012.
- [59] D. T. Frazier, R. Kelly, C. Drovandi, and D. J. Warne, *The Statistical Accuracy of Neural Posterior and Likelihood Estimation*, Nov. 2024.
- [60] A. N. Krouglova, H. R. Johnson, B. Confavreux, M. Deistler, and P. J. Gonçalves, *Multifidelity Simulation-based Inference for Computationally Expensive Simulators*, Feb. 2025.
- [61] Y. Hikida, A. Bharti, N. Jeffrey, and F.-X. Briol, *Multilevel neural simulation-based inference*, Aug. 2025.

- [62] T. P. Prescott, D. J. Warne, and R. E. Baker, “Efficient multifidelity likelihood-free Bayesian inference with adaptive computational resource allocation,” *Journal of Computational Physics*, vol. 496, p. 112 577, Jan. 2024.
- [63] T. P. Prescott and R. E. Baker, “Multifidelity Approximate Bayesian Computation,” *SIAM/ASA Journal on Uncertainty Quantification*, vol. 8, no. 1, pp. 114–138, Jan. 2020.
- [64] D. J. Warne, T. P. Prescott, R. E. Baker, and M. J. Simpson, “Multifidelity multilevel Monte Carlo to accelerate approximate Bayesian parameter inference for partially observed stochastic processes,” *Journal of Computational Physics*, vol. 469, p. 111 543, Nov. 2022.
- [65] D. J. Warne, R. E. Baker, and M. J. Simpson, “Multilevel rejection sampling for approximate Bayesian computation,” *Computational Statistics & Data Analysis*, vol. 124, pp. 71–86, Aug. 2018.
- [66] *Multifidelity multilevel Monte Carlo to accelerate approximate Bayesian parameter inference for partially observed stochastic processes - ScienceDirect*, <https://www.sciencedirect.com/science/article/pii/S0021999122000000>
- [67] M. Chen, F. Li, S. Wang, and Y. Cao, “Stochastic modeling and simulation of reaction-diffusion system with Hill function dynamics,” *BMC Systems Biology*, vol. 11, no. Suppl 3, p. 21, Mar. 2017.
- [68] D. T. Gillespie, “Stochastic Simulation of Chemical Kinetics,” *Annual Review of Physical Chemistry*, vol. 58, no. 1, pp. 35–55, May 2007.
- [69] D. F. Anderson, A. Ganguly, and T. G. Kurtz, “Error analysis of tau-leap simulation methods,” *The Annals of Applied Probability*, vol. 21, no. 6, Dec. 2011.
- [70] M. Matsumoto and T. Nishimura, “Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator,” *ACM Transactions on Modeling and Computer Simulation*, vol. 8, no. 1, pp. 3–30, Jan. 1998.
- [71] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, “Parallel random numbers: As easy as 1, 2, 3,” in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, Seattle Washington: ACM, Nov. 2011, pp. 1–12.
- [72] P. L’Ecuyer, D. Munger, B. Oreshkin, and R. Simard, “Random numbers for parallel computers: Requirements and methods, with emphasis on GPUs,” *Mathematics and Computers in Simulation*, vol. 135, Jun. 2016.
- [73] L. Yu. Barash and L. N. Shchur, “PRAND: GPU accelerated parallel random number generation library: Using most reliable algorithms and applying parallelism of modern GPUs and CPUs,” *Computer Physics Communications*, vol. 185, no. 4, pp. 1343–1353, Apr. 2014.
- [74] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Commun. ACM*, vol. 52, pp. 65–76, Apr. 2009.
- [75] *Performance Optimization on Modern Processor Architecture through Vectorization*, <https://objectcomputing.com/whitepapers/2016-performance-optimization-on-modern-processor-architecture-through-vectorization>.
- [76] *OpenCL - The Open Standard for Parallel Programming of Heterogeneous Systems*, <https://www.khronos.org/opencl/>, Jul. 2013.
- [77] “The OpenCL™ Specification,”
- [78] T. Jahnke and W. Huisinga, “Solving the chemical master equation for monomolecular reaction systems analytically,” *Journal of Mathematical Biology*, vol. 54, no. 1, pp. 1–26, Jan. 2007.
- [79] L. Michaelis and M. L. Menten, “Die Kinetik der Invertinwirkung,” *Biochemische Zeitschrift*, vol. 49, pp. 333–369, 1913.
- [80] R. Marshall and V. Noireaux, “Quantitative modeling of transcription and translation of an all-E. coli cell-free system,” *Scientific Reports*, vol. 9, no. 1, p. 11 980, Aug. 2019.
- [81] M. B. Elowitz and S. Leibler, “A synthetic oscillatory network of transcriptional regulators,” *Nature*, vol. 403, no. 6767, pp. 335–338, Jan. 2000.

- [82] O. Pulkkinen and R. Metzler, “Variance-corrected Michaelis-Menten equation predicts transient rates of single-enzyme reactions and response times in bacterial gene-regulation,” *Scientific Reports*, vol. 5, no. 1, p. 17 820, Dec. 2015.
- [83] F. Wong, A. Dutta, D. Chowdhury, and J. Gunawardena, “Structural conditions on complex networks for the Michaelis–Menten input–output response,” *Proceedings of the National Academy of Sciences*, vol. 115, no. 39, pp. 9738–9743, Sep. 2018.